TYPESCRIPT

PREMCHANDNAIDU.G

Introduction to TypeScript

What is type script?

- TypeScript is an application-scale programming language that provides early access to proposed new JavaScript features and powerful additional features like static type checking. You can write TypeScript programs to run in web browsers or on servers and you can reuse code between browser and server applications.

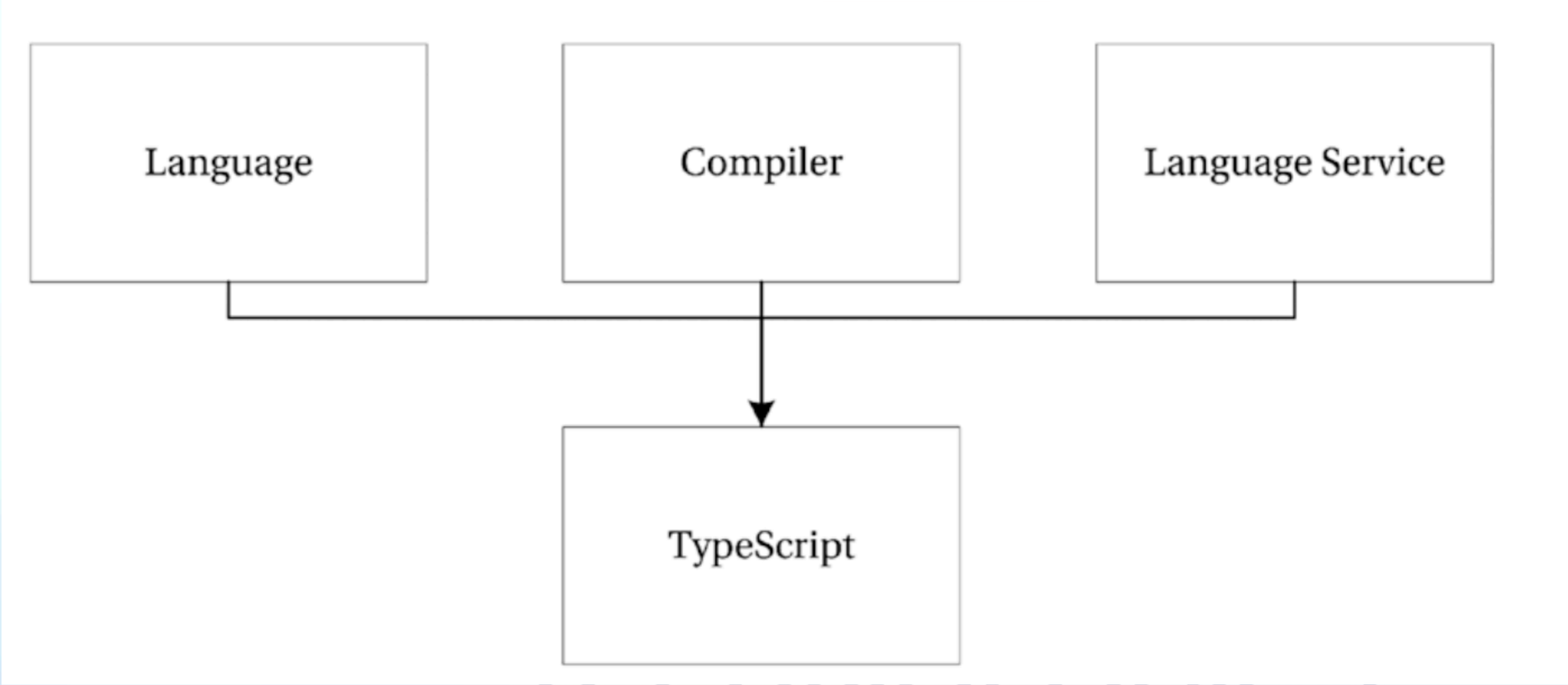
- TypeScript solves many problems in JavaScript, but it respects the patterns and implementation of the underlying JavaScript language, for example, the ability to have dynamic types.

- You can use many integrated development environments with TypeScript, with several providing first-class support including type checking and autocompletion that will improve your productivity and helpeliminate mistakes at design time.

CLAHAN TECHNOLOGIES

- TypeScript is a language, a compiler, and a language service.
- You can paste existing JavaScript into your TypeScript program.
- Compiling from TypeScript to JavaScript is known specifically as transpiling.
- TypeScript is not the only alternative way of writing JavaScript, but it has gainedincredible traction in its first five years.

CLAHAN TECHNOLOGIES


Language
Compiler
Language Service
TypeScript

1. Language

The language part of TypeScript includes its **syntax, keywords, and type system**.
It allows developers to define data types, interfaces, and structures in their programs.
By using types, the programmer clearly describes how data should be used, which helps avoid errors and makes the code easier to understand and maintain.

2. Compiler

The TypeScript compiler is responsible for **converting TypeScript code into JavaScript**.
During this process, it checks the code for type-related errors and gives warnings if it finds any issues.
The compiler removes all type information (type erasure) and can also perform additional tasks such as combining files, generating source maps, and transforming modern syntax into compatible JavaScript.

3. Language Service

The language service provides **intelligent support for developers while writing code**.
It uses the type information from the program to offer features like auto-completion, type hints, error highlighting, and refactoring tools.
This service improves productivity and helps developers write correct and clean code faster.

CLAHAN TECHNOLOGIES

Why typescript is needed?

More Secure
TypeScript adds type safety to JavaScript, which helps prevent common programming mistakes. By catching errors early, it reduces the chances of unexpected behavior in applications, making the code more reliable and secure.

Readable and Maintainable Code
With clear type definitions and structured code, TypeScript makes programs easier to read and understand. This improves maintainability, especially when working in teams or on long-term projects.

GLAHAN TECHNOLOGIES

Early Error Detection
TypeScript detects errors at the **coding stage** by providing warnings, suggestions, and compile-time errors. This helps developers fix problems before the application runs, saving time and effort.

Better Tooling Support
TypeScript offers excellent support in modern editors like VS Code, including auto-completion, type hints, refactoring, and intelligent suggestions, which improve developer productivity.

Suitable for Large-Scale Applications
Because of its strong typing, scalability, and maintainability, TypeScript is widely used in **large and secure applications** such as enterprise systems, financial platforms, and complex web applications.

ELAHAN TECHNOLOGIES

Ts vs js

Feature	JavaScript (JS)	TypeScript (TS)
Type System	Dynamic typing	Static typing
Error Detection	Runtime errors	Compile-time errors
Syntax	Simple, flexible	Slightly stricter
Code Safety	Less safe	More safe due to types
Readability	Harder in large apps	More readable & structured
Maintainability	Difficult for big projects	Easy to maintain
Tooling Support	Basic	Excellent (auto-complete, hints)
Scalability	Not ideal for large apps	Best for large-scale apps
Compilation	Runs directly	Compiled to JavaScript
Learning Curve	Easy to start	Slightly more learning

Js problems->typescript solutions

Each and every value in JavaScript has a set of behaviors you can observe from running different operations. That sounds abstract, but as a quick example, consider some operations we might run on a variable named message.

```
// Accessing the property 'toLowerCase'
// on 'message' and then calling it
message.toLowerCase();
// Calling 'message'
message();
```

CLAHAN TECHNOLOGIES

If we break this down, the first runnable line of code accesses a property called toLowerCase and then calls it. The second one tries to call message directly.

But assuming we don't know the value of message - and that's pretty common - we can't reliably say what results we'll get from trying to run any of this code.

The behavior of each operation depends entirely on what value we had in the first place. Is message callable?

Does it have a property called toLowerCase on it?

If it does, is toLowerCase even callable?

If both of these values are callable, what do they return?

CLAHAN TECHNOLOGIES

Let's say message was defined in the following way.

```
const message = "Hello World!”;
```

As you can probably guess, if we try to run message.toLowerCase(), we'll get the same string only in lower-case.

What about that second line of code? If you're familiar with JavaScript, you'll know this fails with an
exception:

TypeError: message is not a function

It'd be great if we could avoid mistakes like this.
When we run our code, the way that our JavaScript runtime chooses what to do is by figuring out the type of the value - what sorts of behaviors and capabilities it has. That's part of what that

TypeError is alluding to - it's saying that the string "Hello World!" cannot be called as a function.

CLAHAN TECHNOLOGIES

Real world use cases of Type Script

- 1.
- 2.
- 3.
- 4.
- 5.

CLAHAN
TECHNOLOGIES

Steps to install Node js and TypeScript

- Step-1: install node.js from browser
- Step-2: During installation create a folder for installing and writing a code
- Step-3: After installing check these commands
    - Node -v
    - Npm -v
    - Npx -v

CLAHAN TECHNOLOGIES

- Step-4: Installing type script globall
    - Npm install -g typescript
- Step-5: after installing check installation done or not ny using
- Tsc -v
- Step-6: open folder in vs code
- Step-7: create a new .ts file

CLAHAN
TECHNOLOGIES

What is tsc?

`tsc` stands for **TypeScript Compiler**.

It is a **command-line tool** used to **convert TypeScript code into JavaScript**.

What does `tsc` do?

- Compiles `.ts` files into `.js` files
- Checks for **type errors** in the code
- Removes type information (type erasure)
- Shows compile-time **errors and warnings**
- Uses settings from `tsconfig.json`

Why `tsc` is important

Without `tsc`, browsers cannot understand TypeScript.
`tsc` ensures your TypeScript code is **safe, clean, and ready to run as JavaScript**.

CLAHAN TECHNOLOGIES

Lab-1

Task 1: Identify JavaScript Problems

1. Write a small JavaScript example that:
   - Assigns a number to a variable
   - Later assigns a string to the same variable
2. Observe that JavaScript allows this without errors.
3. Note down:
   - What problem this can cause in large application

CLAHAN TECHNOLOGIES

Lab-2

Task 2: Solve the Same Problem Using TypeScript

1. Create a file named:
   `app.ts`
2. Declare variables using **TypeScript types** (`number`, `string`).
3. Try assigning a wrong type and observe the compiler error.

CLAHAN
TECHNOLOGIES

Lab-3

Task 3: TypeScript vs JavaScript Comparison

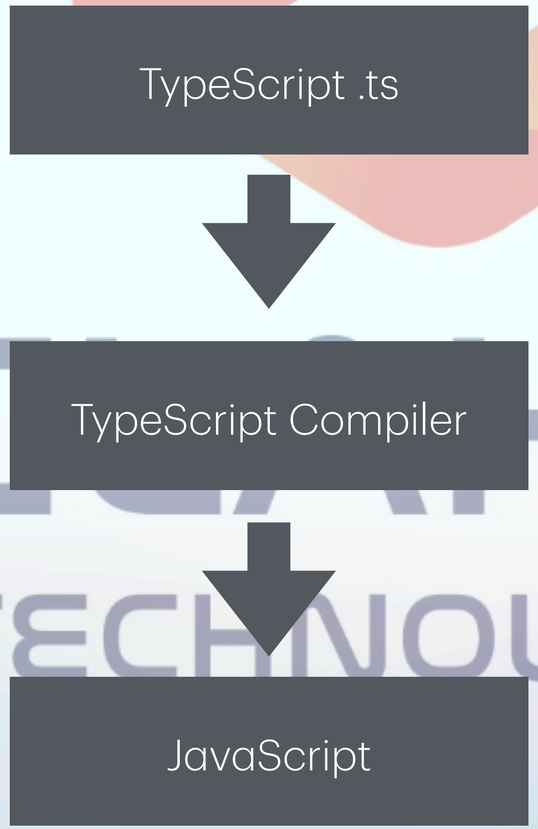
Create a small table or note showing:

- How JavaScript handles types
- How TypeScript handles types
- Which one gives errors earlier and why

CLAHAN TECHNOLOGIES

How TypeScript works (TS → JS)

TypeScript does not run directly in the browser.
First, TypeScript code is **compiled into JavaScript** using the TypeScript compiler.



CLASHAN TECHNOLOGIES

During compilation:

- Types are checked
- Errors are reported
- JavaScript output is generated

CLAHAN
TECHNOLOGIES

Steps to install vite with ts

- Step-1: npx create-vite@latest
- Step-2: project name
- Step-3: package name
- Step-4: select vanilla
- Step-5: select ts
- Step-6: yes

CLAHAN
TECHNOLOGIES

Executing ts file

- Step-1: open integrated terminal
- Step-2: use command tsc filename.ts
- Step-3:After compilation, a javascript file will be generated
    - Filename.ts-> filename.js
- Step-4: Run the generated javascript file by using node.js
    - Node filename.js

CLAHAN TECHNOLOGIES

Type inference

- What is type inference?
- How to implement type inference?
- Where type interface is used?

CLAHAN
TECHNOLOGIES

`tsconfig.json`

`tsconfig.json` is a **configuration file** used by the TypeScript compiler (`tsc`).
It tells TypeScript **how to compile your project**, which files to include, and which rules to follow.

When `tsc` finds a `tsconfig.json`, it treats that folder as a **TypeScript project**

Why `tsconfig.json` is important

- Keeps **consistent compiler settings** across the project
- Enables **strict type checking**
- Controls **output location** and JavaScript version
- Helps in **large and team-based projects**

CLAHAN TECHNOLOGIES

Common `compilerOptions`

Target

The `target` option specifies the **JavaScript version** that the TypeScript compiler should generate.
It ensures that the output JavaScript is compatible with the environment where the code will run, such as older browsers or modern platforms.

Module

The `module` option defines the **module system** used in the generated JavaScript.
It controls how files import and export code, which is important for bundlers and runtime environments like Node.js or browsers.

Strict

The `strict` option enables **strong type-checking rules** across the project.
When enabled, TypeScript performs deeper checks to identify potential errors early, improving code reliability and quality.
It is highly recommended for professional and large-scale applications.

CLAHAN TECHNOLOGIES

outDir

The `outDir` option specifies the **output folder** where all compiled JavaScript files are placed.
This helps keep the source TypeScript files separate from the generated JavaScript, maintaining a clean project structure.

rootDir

The `rootDir` option defines the **main source folder** for TypeScript files.
It tells the compiler where the original code is located so that the folder structure is preserved in the output directory.

CLAHAN
TECHNOLOGIES

Strict mode

- 1.What is Strict mode?
- 2.Why it is important?

CLAHAN
TECHNOLOGIES

Lab-4

- Create a react application using type script
- Create a new tsx component
- Connect to app.js
- Execute conditional rendering example

CLAHAN
TECHNOLOGIES

Basic Data Types

- 1.Primitive data types
- 2.Utility data types
- 3.Collection Data types
- 4.Object based data types

CLAHAN
TECHNOLOGIES

Group	Purpose	Types
Primitive	Single values	number, string, boolean, null, undefined
Special / Utility	Control & safety	any, unknown, void, never
Collection	Multiple values	array, tuple
Structured	Complex data	object, enum

1.primitive data types

Used to store single simple values

- **number** – used to store numeric values
- **string** – used to store text / character values
- **boolean** – used to store `true` or `false` values
- **null** – used to represent an intentional empty value
- **undefined** – used when a variable is declared but not assigned any value

CLAHAN TECHNOLOGIES

2.utility data types

Used for flexibility, safety, or special behavior

- **any**
  → Used when the data type is **not known in advance**, such as **API responses**, **third-party libraries**, or **dynamic user input**
- **unknown**
  → Used when the data type is **unknown but must be checked** before using it (safer than `any`), commonly for **user input** or **external data**
- **void**
  → Used for **functions that perform an action but do not return any value**, such as logging, saving data, or showing alerts
- **never**
  → Used for **functions that never complete normally**, such as **error handling**, **infinite loops**, or functions that always throw an error

CLAHAN TECHNOLOGIES

Explicit vs implicit typing

- Give minimum of 4 differences
- 1.
- 2.
- 3.
- 4.

CLAHAN
TECHNOLOGIES

Arrays, Tuples & Objects

- 1.Arrays:

→ Used to store **multiple values of the same type**

→ Commonly used for **lists** (users, products, marks)

let Students_marks:number =[1,2,3,4,5]

- 2. Tuples

→ Used to store a **fixed number of values with fixed types and order**

→ Commonly used when **structure is known** (id + name, coordinates)

let Students_marks:{number, string, number} =[1,"Ravi",34]

CLAHAN TECHNOLOGIES

Object

→ Used to store **key–value pairs**
→ Used to represent **real-world entities** (user, product, employee)

Let Student:{Id:number,name:String,Age:number}={
Id:1,
Name:”raju”,
Age:34
}

CLAHAN
TECHNOLOGIES

Lab-5

Task 1: Function Return Types & Optional / Default Parameters

- Create a function `greetUser` that takes:
  - `name` (string)
  - Optional `title` (string)
- If `title` is provided, print `"Hello, title name!"`
- Otherwise, print `"Hello, name!"`
- Add a default value for `title` as `"Mr./Ms."`.
- Explicitly define the **return type** of the function.

Task 2: Arrow Functions & Function Type Annotations

- Declare a **function type** `Operation` that takes two numbers and returns a number.
- Implement two arrow functions using this type:
  - `add(a, b)` → returns the sum
  - `multiply(a, b)` → returns the product
- Call both functions and print the results.

CLAHAN TECHNOLOGIES

Function

Functions are the primary means of passing data around in JavaScript. TypeScript allows you to specify the types of both the input and output values of functions.

Parameter Type Annotations

When you declare a function, you can add type annotations after each parameter to declare what types of parameters the function accepts. Parameter type annotations go after the parameter name:

```
// Parameter type annotation
function greet(name: string) {
console.log("Hello, " + name.toUpperCase() + "!!");
}
greet("hi");
```

SIKHAN TECHNOLOGIES

Return type Annotations

You can also add return type annotations. Return type annotations appear after the parameter list:

```
function getFavoriteNumber(): number {
return 26;
}
```

CLAHAN TECHNOLOGIES

Anonymous Functions

Anonymous functions are a little bit different from function declarations. When a function appears
in a place where TypeScript can determine how it's going to be called, the parameters of that function are automatically given types.

```
// No type annotations here, but TypeScript can spot the bug
const names = ["Alice", "Bob", “Eve"];

// Contextual typing for function
names.forEach(function (s) {
console.log(s.toUppercase());

});
```

ELAHAN TECHNOLOGIES

Arrow Functions

```
// Contextual typing also applies to arrow functions
names.forEach((s) => {
console.log(s.toUppercase());

});
```

Even though the parameter s didn't have a type annotation, TypeScript used the types of the forEach function, along with the inferred type of the array, to determine the type s will have.

This process is called contextual typing because the context that the function occurred within informs what type it should have.

CLAHAN TECHNOLOGIES

Object types

Apart from primitives, the most common sort of type you'll encounter is an object type. This refers to any JavaScript value with properties, which is almost all of them! To define an object type, we simply list its properties and their types.

For example, here's a function that takes a point-like object:

```
// The parameter's type annotation is an object type
function printCoord(pt: { x: number; y: number }) {
console.log("The coordinate's x value is " + pt.x);
console.log("The coordinate's y value is " + pt.y);
}
printCoord({ x: 3, y: 7 });
```

TECHNOLOGIES

Optional Properties

Object types can also specify that some or all of their properties are optional. To do this, add a ?after the property name:

```
function printName(obj: { first: string; last?: string }) {
// ...
}
// Both OK
printName({ first: "jagan" });
printName({ first: "jagan", last: "mavayya" });
```

SLAHAN TECHNOLOGIES

Lab-6

Function Return Types

Task 1: Write a function `calculateSquare` that takes a number and returns its square. Explicitly define the return type.

Task 2: Write a function `isEven` that takes a number and returns a boolean indicating whether the number is even. Use TypeScript's type annotations for the return type

CLAHAN TECHNOLOGIES

Lab-7

Optional & Default Parameters

Task 1: Create a function `greetUser` that takes a `name` (string) and an optional `title` (string). If `title` is provided, greet with `"Hello, title name!"`. Otherwise, just `"Hello, name!"`.

Arrow Functions

Task 1: Convert the function `addNumbers(a: number, b: number)` into an arrow function.

CLAHAN TECHNOLOGIES

Literal Types

In addition to the general types string and number, we can refer to specific strings and numbers in type positions.

```
type Students = "santhosh" | "dharithri" | "ajay" | "Thriveni";

let dir: Students;

dir = "santhosh";
```

```
type Marks = 10 | 20 | 30 | 40 | 50 | 60;

function roll(): Marks {
  return 20;
}
```

CLAHAN TECHNOLOGIES

Type Narrowing

Type narrowing is the process where TypeScript refines a *broader type* (like a union) into a *more specific type* based on code logic (usually inside conditionals)

1.typeOf

```
function checkData(x: string | number) {
 if (typeof x === "string") {
   console.log("x is a string");
 } else {
   console.log("x is a number");
 }
}
```

LEAHAN TECHNOLOGIES

2. In operator & Discrimination union

```
type Branch =
  | { type: "CSE"; hari: () => void }
  | { type: "ECE"; ravi: () => void };

function speak(branch: Branch) {
  if (branch.type === "CSE") {
    branch.hari();
  } else {
    branch.ravi();
  }
}
```

```
let branch: Branch;

branch = {
  type: "CSE",
  hari: () => {
    console.log("this is hari");
  }
};

branch = {
    type: "ECE",
    ravi: () => {
        console.log("this is ravi");
    }
};

speak(branch);
speak(branch);
```

Type Aliases

We've been using object types and union types by writing them directly in type annotations. This is convenient, but it's common to want to use the same type more than once and refer to it by a single name.
A type alias is exactly that - a name for any type. The syntax for a type alias is:

```
type Point = {
x: number;
y: number;
};
// Exactly the same as the earlier example
function printCoord(pt: Point) {
console.log("The coordinate's x value is " + pt.x);
console.log("The coordinate's y value is " + pt.y);
}
printCoord({ x: 100, y: 100 });
```

Lab-8

Type Aliases

Task 1: Create a type alias `Point` for an object with `x` and `y` as numbers. Write a function `printPoint` that takes a `Point` and logs it.

Type Aliases

Task 1: Create a type alias `Point` for an object with `x` and `y` as numbers. Write a function `printPoint` that takes a `Point` and logs it.

CLAHAN TECHNOLOGIES

Lab-9

Literal Types

Task 1: Create a function `move` that accepts a direction: `"up" | "down" | "left" | "right"`. Log `"Moving direction"`.

Type Narrowing

Task 1: Write a function `formatValue` that accepts `value: string | number`. If it's a string, print uppercase. If number, print rounded value.

CLAHAN TECHNOLOGIES

Interfaces

An interface declaration is another way to name an object type

```
 type Branch =
  | { type: "CSE"; hari: () => void }
  | { type: "ECE"; ravi: () => void };

function speak(branch: Branch) {
  if (branch.type === "CSE") {
    branch.hari();
  } else {
    branch.ravi();
  }
}

 interface Point {
x: number;
y: number;
}
function printCoord(pt: Point) {
console.log("The coordinate's x value is " + pt.x);
console.log("The coordinate's y value is " + pt.y);
}
printCoord({ x: 100, y: 100 });
```

CLAHAN TECHNOLOGIES

TypeScript does not care whether you define a shape using an **interface** or a **type alias**. In both cases, it only checks whether the value passed to a function has the required properties.

When a value is used, TypeScript compares its **structure** (the properties and their types), not the **name** of the type. If the structure matches what is expected, the code is considered valid—even if the value comes from a different type or has extra properties.

Because TypeScript focuses only on the structure and capabilities of values, it is called a **structurally typed type system**. This means compatibility is determined by “what an object looks like” rather than “what it is called.”

CLAHAN TECHNOLOGIES

Interface

Extending an interface

```
interface Animal {
  name: string;
}

interface Bear extends Animal {
  honey: boolean;
}

function getBear(): Bear {
  return {
    name: "Baloo",
    honey: true
  };
}

const bear = getBear();

console.log(bear.name);
console.log(bear.honey);
```

Type

Extending a type via intersections

```
type Animal = {
  name: string;
};

type Bear = Animal & {
  honey: boolean;
};

function getBear(): Bear {
  return {
    name: "Baloo",
    honey: true
  };
}

const bear = getBear();

console.log(bear.name);
console.log(bear.honey);
```

CLAHAN TECHNOLOGIES

Optional & Readonly Properties

Optional properties (`?`) mean the property *may or may not* be present.

Readonly properties (`readonly`) can be set once but **cannot be changed later**.

They help make objects safer and more predictable.

```
interface User {
  readonly id: number;
  name: string;
  age?: number;
}

const user: User = {
  id: 1,
  name: "Hari"
};

 user.id = 2;
user.name = "Ravi";
```

CLAHAN TECHNOLOGIES

Interface with functions

Interfaces can define **function signatures**.

They describe **what the function should accept and return**, not the implementation.

```
interface Calculator {
  add(a: number, b: number):number;
}

const calc: Calculator = {
  add(a, b) {
    return a + b;
  }
};
```

ELAHAN TECHNOLOGIES

Interface with Arrays

Interfaces can describe **array structures**.

Useful when working with lists of values or objects.

```
interface StringArray {
  [index: number]: string;
}

const names: StringArray = ["Hari", "Ravi", "Sita"];
```

CLAHAN TECHNOLOGIES

Interface with Array Objects

```typescript
interface Student {
  name: string;
  marks: number;
}

const students: Student[] = [
  { name: "Hari", marks: 90 },
  { name: "Ravi", marks: 85 }
];
```

TECHNOLOGIES

Lab-10

Task 1: Interface Basics & Optional / Readonly Properties

Create an interface `User` with:

- `id` (readonly)
- `name` (string)
- `age` (optional number)

Create an object of type `User` and print its properties. Try to modify `id` and observe the error.

Task 2: Interface with Functions

Create an interface `Calculator` with two methods:

- `add(a: number, b: number): number`
- `multiply(a: number, b: number): number`

Implement this interface in an object and call both methods.

ELAHAN TECHNOLOGIES

Lab-11

Task 3: Interface with Arrays

Create an interface `StudentList` representing an array of objects `{ name: string; marks: number; }`.
Create an array of 3 students and print the name and marks of each student.

Task 4: Interface vs Type & Combined Practice

- Create a **type alias** `PointType` with `x` and `y` numbers.
- Create an **interface** `PointInterface` with `x` and `y` numbers.
- Write a function `printPoint` that accepts both and prints `x` and `y`.
- Bonus: Explain in a comment the difference between `type` and `interface`.

CLAHAN TECHNOLOGIES

Type Assertions

Sometimes you will have information about the type of a value that TypeScript can't know about.
They are used when TypeScript cannot automatically infer a type, but **you are sure** about it.
Type assertions **do not change the runtime value** — they only affect the **type system**.

```
let value: unknown = "Hello";

let str = value as string;
```

CLAHAN TECHNOLOGIES

Why Type Assertions Are Needed

Working with Unknown

TypeScript is **safe by default**.
For `unknown` or union types, it blocks access until the type is confirmed.

```
let value: unknown = "Hello";

value.toUpperCase();
```

```
let str = value as string;

console.log(str.toUpperCase());
```

CLAHAN TECHNOLOGIES

Woking with Dom

TypeScript can’t know which DOM element it is — you tell it.

```
const input = document.getElementById("username") as HTMLInputElement;
input.value = "Hari"
```

CLAHAN
TECHNOLOGIES

Enums

Enums are a feature added to JavaScript by TypeScript which allows for describing a value which could be one of a set of possible named constants.

Unlike most TypeScript features, this is not a type-level addition to JavaScript but something added to the language and runtime.

Because of this, it’s a feature which you should know exists, but maybe hold off on using unless you are sure.

CLAHAN TECHNOLOGIES

When to use Enum

Enums define a set of named constants; numeric enums auto-increment, while string enums provide clearer, safer values. Enums should be used when representing fixed, related states.

Use enums when:

- You have a **fixed set of related values**
- Values should not change
- You want **readability and safety**
- You want to avoid **magic strings/numbers**

CLAHAN TECHNOLOGIES

Numeric Enum

```
enum OrderStatus {
  pending,//0
  transit,//1
  out_for_delivery,//2
  success,//3
}

OrderStatus.pending;
```

```
interface Order {
  id: number;
  product: string;
  status: OrderStatus;
}
```

CLAHAN TECHNOLOGIES

```typescript
const order1: Order = {
  id: 101,
  product: "Laptop",
  status: OrderStatus.pending
};

const order2: Order = {
  id: 102,
  product: "Mobile",
  status: OrderStatus.success
};
```

CLAHAN TECHNOLOGIES

```typescript
function getOrderMessage(status: OrderStatus): string {
  switch (status) {
    case OrderStatus.pending:
      return "Order placed, waiting for pickup";

    case OrderStatus.transit:
      return "Order is on the way";

    case OrderStatus.out_for_delivery:
      return "Out for delivery";

    case OrderStatus.success:
      return "Delivered successfully";

    default:
      return "Unknown status";
  }
}
```

CLAHAN TECHNOLOGIES

String Enum

```
enum Status {
  Success = "SUCCESS",
  Failure = "FAILURE"
}

Status.Success; // "SUCCESS"
```

CLAHAN
TECHNOLOGIES

Lab-12

Task 1: Numeric & String Enums

1. Create a **numeric enum** `Direction` with values `Up`, `Down`, `Left`, `Right`.
2. Create a **string enum** `Status` with values `"PENDING"`, `"SUCCESS"`, `"FAILED"`.
3. Write a function `printStatusOrDirection` that accepts either enum and logs a message.

Bonus: Assign enum values to variables and print them.

Task 2: Type Assertions & DOM Type Casting

1. In an HTML page, create an `<input>` element with id `"username"`.
2. In TypeScript, get the input element and cast it to `HTMLInputElement` using `as`.
3. Write a function `printInputValue` that reads the input's value and logs it.

Note: Use a union type or `unknown` value and assert it to a specific type.

CLAHAN TECHNOLOGIES

Classes

TypeScript offers full support for the class keyword introduced in ES2015.

As with other JavaScript language features, TypeScript adds type annotations and other syntax to allow you to express relationships between classes and other types.

Class Members

```
Class student{}
```

CLAHAN TECHNOLOGIES

Fields

- A field declaration creates a public writeable property on a class

```
class student{
    name:string;
    age:number;
}

let st=new student();
st.name="raju";
st.age=24;
```

CLAHAN TECHNOLOGIES

Constructor

```
class student{
    constructor(public name:string, public age:number){}
}
let st=new student("raju",24);


console.log(st.name)
```

ELAHAN TECHNOLOGIES

Parameterized constructor using this keyword

```
class student{
    name:string;
    age:number;
    constructor(public Student_name:string, public Student_age:number){
        this.name=Student_name;
        this.age=Student_age;
    }
}
let st=new student("raju",24);
console.log(st.age)
```

ELAHAN TECHNOLOGIES

Readonly

```
class Greeter {
readonly name: string = "world";

}

const g = new Greeter();
g.name = "also not ok";
```

CLAHAN TECHNOLOGIES

Access modifiers

You can use TypeScript to control whether certain methods or properties are visible to code outside the class.

public
The default visibility of class members is public. A public member can be accessed anywhere:

protected
protected members are only visible to subclasses of the class they're declared in

private
private is like protected, but doesn't allow access to the member even from subclasses:

CLAHAN TECHNOLOGIES

Modifier	Can be accessed from:	Notes
`public`	anywhere (default)	No restriction
`private`	only inside the class	Not accessible from outside
`protected`	inside class **and subclasses**	Useful for inheritance

TECHNOLOGIES

Public

```
class Student {
  public name: string;
  constructor(name: string) {
    this.name = name;
  }
}

let st = new Student("Raju");
console.log(st.name);
```

CLAHAN TECHNOLOGIES

Private

```
class Student {
  private age: number;
  constructor(age: number) {
    this.age = age;
  }

  showAge() {
    console.log(this.age);
  }
}

let st = new Student(24);
st.showAge();
st.age;
```

CLAHAN TECHNOLOGIES

Protected

```
class Student {
  protected id: number;
  constructor(id: number) {
    this.id = id;
  }
}

class CollegeStudent extends Student {
  showId() {
    console.log(this.id);
  }
}

let st = new CollegeStudent(101);
st.showId();
```

CLAHAN TECHNOLOGIES

Inheritence

```
class student{
   student_name:string="hari";
}

class teacher  extends student{
   teacher_name="raju"
}

let t=new teacher();
console.log(t.student_name);
console.log(t.teacher_name);
```

CLAHAN TECHNOLOGIES

Lab-13

Task 1: Basic Class & Constructor

- Create a class `Student` with properties `name` (string) and `age` (number).
- Initialize the properties using a **constructor**.
- Create an instance of `Student` and print the values of `name` and `age`.

Task 2: Access Modifiers

- Add **`private`** property `rollNo` and **`public`** property `name` to the `Student` class.
- Write a method `showRollNo()` inside the class to print `rollNo`.
- Try accessing `rollNo` outside the class and note the TypeScript error.
-

ELAHAN TECHNOLOGIES

Lab-14

Task 3: Readonly Property

- Add a **readonly** property `id` to the `Student` class.
- Assign a value to `id` in the constructor.
- Try modifying it outside the class and observe the TypeScript error.

Task 4: Inheritance

- Create a parent class `Animal` with property `name` and method `makeSound()`.
- Create a child class `Dog` that **extends** `Animal` and adds a property `breed`.
- Override `makeSound()` in `Dog` to print `"Woof! Woof!"`.
- Create instances of both classes and call their methods.

CLAHAN TECHNOLOGIES

ES6 Features with TypeScript

- let / const
- Arrow functions
- Destructuring
- Spread operator
- import / export

CLAHAN
TECHNOLOGIES

Lab15 (Final Project)

1. **Project Setup**
   - Create a new React project with TypeScript.
2. **Navbar Component**
   - Create a Navbar with links: Home, Services, Contact Us.
3. **Pages**
   - Create **Home Page** with welcome message and an image.
   - Create **Services Page** listing at least 3 services.
   - Create **Contact Us Page** with a form: Name, Email, Message, Submit button.
4. **Routing**
   - Set up routing between Home (`/`), Services (`/services`), and Contact Us (`/contact`).
5. **Design & Styling**
   - Add some CSS design to Navbar and pages.
   - Include at least 1 image on each page.
6. **TypeScript Practice**
   - Use interfaces/type annotations for component props.
   - Type the form inputs and any props in components.
7. **Optional Challenge**
   - Make Navbar responsive.
   - Validate form inputs with TypeScript.
   - Add a Footer component with social media links

CLAHAN TECHNOLOGIES